

程序设计

讲义

2026 年秋季

毕朝日

复旦大学集成电路与微纳电子创新学院

`zhaori_bi@fudan.edu.cn` `https://bzs2915.github.io/bzs2915/`

目录

第三讲 字符串与格式化输入输出	4
3.1 一串字符怎样放进内存	4
3.1.1 字符数组与下标	4
3.1.2 引号、空字符与空字符串	5
3.1.3 修改一个字母与改变结束位置	6
3.2 数组大小与字符串长度	7
3.2.1 sizeof 测量对象所占空间	7
3.2.2 strlen 沿字符串寻找结束字符	8
3.2.3 字节数与屏幕上的字符数	9
3.3 给常用的数值和文字命名	9
3.3.1 从体积计算中的 62.4 说起	9
3.3.2 等号和分号为什么会出错	10
3.3.3 const 声明只读对象	10
3.4 姓名、体重与一条计算结果	11
3.4.1 读入一个名字	11
3.4.2 为什么 name 前没有取地址符	13
3.4.3 赞语有多少字节	13
3.5 把输出排成读得清楚的文字	14
3.5.1 格式字符串怎样使用参数	14
3.5.2 字段宽度规定最少占多少位置	16
3.5.3 同一个精度数字有不同含义	17
3.5.4 标志怎样组合	18
3.5.5 字符串的宽度与输出上限	20
3.5.6 一条长语句怎样分行书写	20
3.6 格式、类型与返回值	21
3.6.1 长度修饰符并不规定固定字节数	21
3.6.2 同一个整数为什么能显示成字符	22
3.6.3 缺参数与错类型没有可靠的预测输出	23
3.6.4 printf 返回多少	24

3.7	格式化输入的读取过程	25
3.7.1	格式说明、地址与赋值	25
3.7.2	一次成功赋值后，后面还有什么	26
3.7.3	普通字符必须在相应位置匹配	28
3.7.4	字符输入为什么会读到换行	28
3.7.5	输入宽度规定最多接收多少	29
3.8	星号在输入与输出中各做什么	31
3.8.1	输出时由参数提供宽度和精度	31
3.8.2	输入时跳过项目而不赋值	32
3.9	一次只读写一个字符	33
3.9.1	getchar 的返回值为什么用 int 接收	33
3.9.2	与格式化字符读写的对应	33
3.10	习题与解析	34
3.10.1	习题一：预测文字、字符和数字的输出	34
3.10.2	习题二：修正姓名和年龄程序	35
3.10.3	习题三：补出带引号和百分号的输出	36
3.10.4	习题四：由显示要求写出转换说明	37
3.10.5	习题五：为输入选择对象和地址	39
3.10.6	习题六：数组容量、内容和终止位置	39
3.10.7	习题七：宽度、精度和补零怎样配合	40
3.10.8	习题八：在输入中标出停止位置	40
3.10.9	习题九：两个星号和一个返回值	41
3.10.10	习题十：逐项检查接口是否匹配	41
3.11	本讲总结	43

第三讲 字符串与格式化输入输出

1972 年 1 月，汤普森在 PDP-11 版 B 语言手册中规定：字符串的字符连续存放，末尾加上一个特殊的结束标记，写成 `*e`。程序拿到字符串的起始位置后，就可以沿着内存逐个读取字符，直到遇见这个标记。它解决了一个具体问题：知道文字从哪里开始之后，还要知道文字到哪里结束。

第二讲讨论过 B 怎样把几个字符装进一个机器字，以及 C 的 `char` 类型怎样让编译器直接定位单个字节。能够逐字节访问以后，结束位置仍然需要约定。C 字符串以数值为 0 的空字符结束，在源代码中写成 `'\0'`。例如，保存名字 `Morgan` 需要六个字母，再加一个结束字符，共七个字节。

若程序预留了 40 个字节来存名字，输入 `Morgan` 之后，这块空间仍有 40 个字节，名字的长度却是 6。输出函数沿着字符找到空字符就停止，不会为了用完 40 个字节，把后面的内容也打印出来。数组提供了多少空间、其中存了多少文字，是两个不同的问题。

名字还会和数字一起出现。一个登记程序可能先问姓名和体重，再输出一行计算结果；一个书目程序可能把书名与价格排成表格。名字、输入的数字和屏幕上的结果，都经过字符形式的输入或输出，程序必须分别处理存储边界、类型转换和显示格式。

3.1 一串字符怎样放进内存

3.1.1 字符数组与下标

保存单个字母可以定义 `char ch;`。保存整个名字需要一组连续的字符元素，这样的一组元素称为字符数组：

```
char name[40];
```

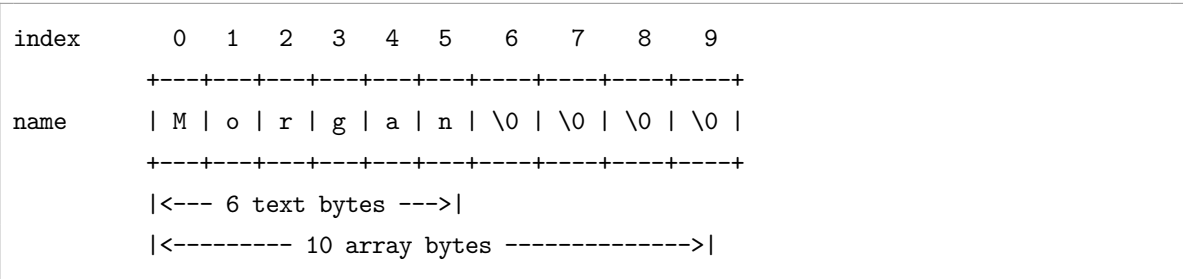
`name` 是数组名，方括号中的 40 规定元素个数，每个元素的类型都是 `char`。C 中 `sizeof(char)` 恒为 1，所以整个数组占 40 个 C 字节。一个 C 字节包含多少位由实现规定；本讲涉及 ASCII 编码数值的例子采用常见的 8 位字节环境。

数组元素用下标编号，下标从 0 开始。`name[0]` 表示第一个元素，`name[1]` 表示第二个元素，最后一个是 `name[39]`。`name[40]` 已经超出这个数组的范围，不能用来存第 40 个字符。

在定义数组时可以给出初值。下面的定义把名字存入数组，并由编译器补上结束字符：

```
char name[10] = "Morgan";
```

图中每个格子是一个 char 元素。\\0 表示该格保存数值 0，虽然源代码用了反斜杠和数字 0 两个符号，内存中仍只有一个字符。



第 0 至 5 号元素保存六个字母，第 6 号元素标记字符串结束。数组在定义时只给出部分初值，其余元素也会初始化为 0，因此第 7 至 9 号元素同样为 0。字符串处理函数遇到第 6 号元素就已经停止，后面三个 0 不增加字符串长度。

这条初始化规则与 char name[40]; 不同。若后一个定义位于 main 函数内部，且没有其他初始化，数组内容不会自动清零。在输入函数成功写入之前，不能把其中未初始化的数据当作字符串读取。

3.1.2 引号、空字符与空字符串

字符串是以第一个空字符结束的一段连续字符序列，结束字符属于它的存储表示，但不计入通常所说的字符串长度。C 没有专用的基本字符串类型，普通字符串通常保存在 char 数组中。双引号包围的"Morgan" 称为字符串字面量；两端的双引号是源代码的界定符，不保存进这段文字。

表 3.1: 容易混淆的字符与字符串写法

写法	含义
'A'	普通字符常量，C 中类型为 int；ASCII 编码值为 65。
"A"	字符串字面量，包含'A' 与'\\0' 两个 char 元素。
'0'	数字字符 0；在 ASCII 中编码值为 48。
'\\0'	数值为 0 的空字符，用于标记字符串结束。
' '	空格字符；在 ASCII 中编码值为 32。
""	空字符串，长度为 0，存储时仍需一个空字符。

空格可以出现在文字中。例如，"This is a book" 的相邻单词之间各有一个空格，空格也计入长度；该字符串一共 14 个内容字节，加上结束字符需要 15 个字节。"Hello!\\n" 包含六个可见字符和一个换行字符，长度为 7，存储时需要 8 个字节。

数组是否保存了字符串，要看内容是否满足结束约定。下面两个数组都有三个元素：

```
char word[3] = {'H', 'i', '\0'};
char raw[3]  = {'H', 'i', '!'};
```

`word` 保存字符串 "Hi"。`raw` 保存三个字符，但其中没有空字符。若直接把 `raw` 交给 `strlen` 或不带精度限制的 `printf("%s", raw)`，函数寻找结束位置时会越过数组边界，产生未定义行为。未定义行为表示 C 语言不保证程序接下来怎样运行，不能把某次恰好打印出文字当作代码正确的证据。

3.1.3 修改一个字母与改变结束位置

数组初始化后，可以给其中的单个元素赋值。下面的完整程序先修改第一个字母，再把结束位置移到下标 3 处：

```
#include <stdio.h>

int main(void) {
    char name[10] = "Morgan";

    printf("%s\n", name);
    name[0] = 'J';
    printf("%s\n", name);
    name[3] = '\0';
    printf("%s\n", name);
    return 0;
}
```

程序输出：

```
Morgan
Jorgan
Jor
```

`%s` 要求 `printf` 从给定位置输出字符串，遇到第一个空字符时停止。执行 `name[0] = 'J'`；之后，首字母变为 J，其余内容未变；执行 `name[3] = '\0'`；之后，第 3 号元素从 g 变为空字符，于是只输出前面的 J、o、r。

此时，第 4、5 号元素中的 a 和 n 仍保留在数组里，整个数组仍占 10 字节。字符串提前结束，并不意味着数组缩小，也不意味着结束字符之后的内存全部被擦除。

前面的数组定义用字符串字面量给各元素赋了初值。定义完成后，下面这种对整个数组赋值的写法不合法：

```
name = "Jordan";
```

数组元素赋值与整段文字复制是不同操作。需要修改本例中的首字母时，使用 `name[0]` 即可。初始化得到的字符数组可以修改，字符串字面量本身则不应被修改。

3.2 数组大小与字符串长度

3.2.1 sizeof 测量对象所占空间

`sizeof` 是求对象或类型大小的运算符。对上节的数组求 `sizeof name`，得到整个数组所占的字节数 10；内容从 `Morgan` 改为 `Jor`，数组类型和元素个数都没有变化，结果仍为 10。

```
sizeof name
sizeof(name)
sizeof(char)
```

前两种写法都对表达式 `name` 求大小，括号可以省略。最后一种对类型名 `char` 求大小，括号必须保留。`sizeof` 不是函数，书写时不能据是否有括号来判断它的性质。

大小用 `size_t` 类型表示，这是标准库用于表示对象大小的无符号整数类型。输出 `size_t` 用 `%zu`：z 指定相应的大小类型，u 指定无符号十进制输出；z 不代表固定的字节数。

```
#include <stdio.h>

int main(void) {
    char name[40] = "Morgan";

    printf("name: %zu bytes\n", sizeof name);
    printf("char: %zu byte\n", sizeof(char));
    printf("int: %zu bytes\n", sizeof(int));
    printf("float: %zu bytes\n", sizeof(float));
    return 0;
}
```

在 `int` 和 `float` 均占 4 字节的环境中，输出为：

```
name: 40 bytes
char: 1 byte
int: 4 bytes
float: 4 bytes
```

前两个结果由本例的数组定义和 C 的 `char` 大小规则确定，后两个结果依赖实现。对本讲这些固定大小的数组，编译器根据类型就能确定大小，无须逐个读取内容。

3.2.2 `strlen` 沿字符串寻找结束字符

`strlen` 是 `<string.h>` 声明的字符串长度函数，名字可读作 `string length`。它从字符串开头数到第一个空字符，返回之前的字节数，返回类型同样是 `size_t`。

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char name[40] = "Morgan";
    char greeting[] = "Hello!\n";
    char hidden[] = "AB\OCD";

    printf("name: %zu %zu\n", strlen(name), sizeof name);
    printf("greeting: %zu %zu\n",
           strlen(greeting), sizeof greeting);
    printf("hidden: %zu %zu\n",
           strlen(hidden), sizeof hidden);
    printf("empty: %zu %zu\n", strlen(""), sizeof "");
    return 0;
}
```

每行先输出字符串长度，再输出数组大小：

```
name: 6 40
greeting: 7 8
hidden: 2 6
empty: 0 1
```

`char greeting[] = "Hello!\n";` 没有写元素个数，编译器便根据初始内容确定大小，七个内容字节加上一个结束字符，共 8 字节。`hidden` 依次保存 A、B、空字符、C、D，以及字面量末尾自动补入的空字符，共 6 字节；`strlen` 在第一个空字符处停止，因此返回 2。

```
hidden:  +---+---+---+---+---+
          | A | B | \0 | C | D | \0 |
          +---+---+---+---+---+
strlen:   1  2  stop
sizeof:  <----- 6 bytes ----->
```

只有数组恰好容纳一段不含内嵌空字符的文字及其结束字符时，数组大小才等于 `strlen` 的结果加 1。`name[40]` 和 `hidden` 都说明，不能把 `sizeof s == strlen(s) + 1` 当作普遍规律。

数组传参后还能测到大小吗（选读）

将数组作为普通函数实参时，通常传递的是指向首元素的指针，即保存首元素地址的值。例如，`strlen(name)` 得到这个起点，再读取字符以寻找空字符；它没有自动收到数组容量 40。若在函数参数表里写 `char s[]`，该参数会按字符指针处理，在函数内对参数 `s` 求 `sizeof`，得到的是指针大小。`sizeof name` 直接作用于本节定义的数组时，数组不会作这种转换，所以结果为 40。需要在函数中使用容量，应另外传入容量值。

3.2.3 字节数与屏幕上的字符数

第二讲用 UTF-8 说明过“中”占三个字节。若源代码中的普通字符串按 UTF-8 编码存储，“A 中”的内容字节为：

```
visible text      A          (Chinese character)
UTF-8 bytes      41          E4  B8  AD          00
                  |<----- 4 text bytes ----->| end
```

因此，`strlen("A 中")` 为 4，字面量数组大小为 5，屏幕上却是两个可见字符。`strlen` 按字节计数，不负责理解 UTF-8 字符边界。后面用输出精度限制字符串时也要考虑这一点：截到一个汉字的中间，会留下不完整的编码。为使宽度与长度的推导清楚，本讲的格式对齐实验使用基本英文字符。

3.3 给常用的数值和文字命名

3.3.1 从体积计算中的 62.4 说起

假设按密度 62.4lb/ft³ 估算一个物体的体积。lb 表示磅，ft 表示英尺，ft³ 表示立方英尺。输入以磅计的质量，除以该密度，就得到以立方英尺计的体积。本例沿用日常体重记录的单位与英文变量名 `weight`；62.4 是计算模型选定的近似值。

直接写 `volume = weight / 62.4;` 能够计算，却没有说明 62.4 的用途。用名字 `DENSITY` 表示密度，计算式就成为 `volume = weight / DENSITY;`。当同一数值用于多处时，统一的名字也便于修改。

预处理器在编译器进行通常的语法、类型分析之前处理以 `#` 开头的指令。`#define` 可以定义一个宏，为一段待替换的内容命名。用宏表示固定数值或固定文字，是定义符号常量的一种常见方式：

```
#define DENSITY 62.4
#define PRAISE "What a super marvelous name! "
#define NEWLINE '\n'
```

随后出现的宏名会在预处理时被替换。`weight / DENSITY` 展开为 `weight / 62.4;`；`PRAISE` 展开为那段双引号包围的文字，末尾感叹号后还有一个空格。宏名采用大写是便于识别的风格约定，命名仍遵守标识符规则。

3.3.2 等号和分号为什么会出错

宏定义中，宏名后面的内容就是替换内容。如果写成：

```
#define TOES = 10
digits = fingers + TOES;
```

展开结果为：

```
digits = fingers + = 10;
```

多出的等号使赋值语句不合法。类似地，`#define TOES 10;` 把分号也放进了替换内容；`digits = TOES + 2;` 会展开成 `digits = 10; + 2;`，已经不是原先要写的加法。定义数值常量应写成 `#define TOES 10`。

替换也会暴露运算符优先级问题：

```
#define BASE 2 + 3
int count = BASE * 4;
```

展开后的表达式是 `2 + 3 * 4`，乘法先做，结果为 14。若希望宏代表和 5，应写 `#define BASE (2 + 3)`，使展开结果为 `(2 + 3) * 4`，得到 20。宏展开后仍须按 C 表达式的规则求值，名字本身不会替表达式添加括号。

3.3.3 `const` 声明只读对象

另一种办法是定义带有 `const` 限定的对象。`const` 表示程序不能通过这个名字给对象重新赋值：

```
const double density = 62.4;
const int toes = 10;
```

`density` 有明确类型 `double`，定义时获得初值 62.4；以后若写 `density = 63.0;`，编译器会报告试图修改只读对象。定义末尾的分号属于正常的 C 声明。

表 3.2: 数值宏与 `const` 对象

	<code>#define TOES 10</code>	<code>const int toes = 10;</code>
作用方式	预处理时替换宏名	定义有类型的只读对象
类型	宏名无独立类型；展开后的 10 有类型	对象类型为 <code>const int</code>
等号、分号	不写入此数值宏的定义	按初始化声明书写
用作 C 整数常量表达式	此例展开为整数常量，可以	此例的对象名一般不可以

整数常量表达式是 C 对某些编译期用途规定的一类表达式，例如由整数常量参与允许的运算构成。`const` 保证只读，却不使对象名在 C11 中自动成为整数常量表达式。因此不能把 `const int n = 40;` 与 `#define N 40` 在所有场合中互换。本讲的固定字符数组直接写出容量，避免把字符串处理与变长数组问题混在一起。

3.4 姓名、体重与一条计算结果

3.4.1 读入一个名字

现在让使用者输入名字，再计算体积。名字保存在 40 字节数组中，体重用 `double` 保存。`scanf` 的 `%s` 转换读取一个由非空白字符组成的单词；在 `s` 前写 39，表示最多接收 39 个内容字节，给末尾的空字符留一个位置。

```
#include <stdio.h>
#include <string.h>
#define DENSITY 62.4

int main(void) {
    char name[40];
    double weight, volume;

    printf("Hi! What's your first name?\n");
    if (scanf("%39s", name) != 1) {
        return 1;
    }
    printf("%s, what's your weight in pounds?\n", name);
    if (scanf("%lf", &weight) != 1) {
        return 1;
    }
    volume = weight / DENSITY;
    printf("Well, %s, your volume is %.2f cubic feet.\n",
           name, volume);
    printf("Name length: %zu bytes; capacity: %zu bytes.\n",
           strlen(name), sizeof name);
    return 0;
}
```

先输入 Morgan，再输入 156，最后两行输出为：

```
Well, Morgan, your volume is 2.50 cubic feet.
Name length: 6 bytes; capacity: 40 bytes.
```

除法 $156/62.4 = 2.5$ ，`%.2f` 使输出保留两位小数。第二讲已经解释过，输出精度改变文字的显示形式，不会把 `volume` 内部保存的值改写成另一个精度的对象。

程序中两个 `if` 沿用第二讲的输入检查方法：`scanf` 每次应成功赋值一项，返回值不等于 1 就结束本次运行。体积计算假定读入的是类型范围内的合理正数；成功转换一个数值本身，并不检查体重是否具有实际意义。

3.4.2 为什么 name 前没有取地址符

`scanf("%lf", &weight)` 需要 `weight` 的地址, 以便向这个 `double` 对象写入数值。`scanf("%39s", name)` 同样需要地址, 只是数组名 `name` 在这里会转换为首元素地址, 无须再写 `&`。

```
input: Morgan\n

name -----> +---+---+---+---+---+---+---+---+---+
                | M | o | r | g | a | n | \0 |   ...   |
                +---+---+---+---+---+---+---+---+---+
                0   1   2   3   4   5   6           39

input: 156\n

&weight ----> +-----+
                | double value: 156.0 |
                +-----+
```

字符串转换从首元素开始依次写入多个字节, 再写入空字符; 数值转换向 `weight` 这个对象写入一个浮点数。`name` 在本次调用中的含义等同于 `&name[0]`。`&name` 表示整个数组的地址, 类型不同, 不能因为起始位置看起来相同, 就把它当成 `%s` 所需的字符指针。

读取 `Morgan` 后, 输入行末的换行仍未被 `%s` 读取。随后 `%lf` 会跳过这个换行, 再读体重, 所以这两个调用能够连续配合。若输入的是 `Morgan Lee`, `%39s` 只取 `Morgan`, 剩下的 `Lee` 会使后面的数值转换失败。程序约定输入一个名字单词, 不能把含空格的全名直接套进这一输入方式。

长度限制防止该次字符串写入越界, 但不会自动处理超长单词。若输入超过 39 个内容字节, 未读部分仍留在输入中, 可能影响下一次读取。数组容量、单词分隔和后续输入都要一起考虑。

3.4.3 赞语有多少字节

宏 `PRAISE` 代表固定文字, 可以同时用于输出、长度计算和大小计算:

```
#include <stdio.h>
#include <string.h>
#define PRAISE "What a super marvelous name! "

int main(void) {
    char name[40];

    printf("What's your name?\n");
    if (scanf("%39s", name) != 1) {
        return 1;
    }
    printf("Hello, %s. %s\n", name, PRAISE);
    printf("Name: length %zu, array size %zu.\n",
        strlen(name), sizeof name);
    printf("Praise: length %zu, array size %zu.\n",
        strlen(PRAISE), sizeof PRAISE);
    return 0;
}
```

输入 Morgan 后，长度报告是：

```
Name: length 6, array size 40.
Praise: length 29, array size 30.
```

赞语包含 24 个非空格字符和 5 个空格，共 29 个内容字节。最后一个空格位于感叹号后面，也被 `strlen` 计数。`sizeof PRAISE` 在宏展开后成为对字符串字面量求大小，所以连同结束字符得到 30；它没有测量宏名占几个字母。

3.5 把输出排成读得清楚的文字

3.5.1 格式字符串怎样使用参数

`printf` 的第一个参数是**格式字符串**。其中的普通字符按原样输出，以%开始的转换说明规定怎样处理后面的参数。用一条语句输出概率和成绩：

```
#include <stdio.h>

int main(void) {
    int imate = 2;
    long shot = 53456L;
    char grade = 'A';
    float score = 2.71828f;

    printf("The odds against the %d were %ld to 1.\n",
           imate, shot);
    printf("A score of %f is not an %c grade.\n",
           score, grade);
    return 0;
}
```

输出为:

```
The odds against the 2 were 53456 to 1.
A score of 2.718280 is not an A grade.
```

第一个%d 使用 imate, %ld 使用 shot。第二个调用中, float 的值在传参时提升为 double, 由%f 输出; grade 的值在当前环境下提升为 int, 由%c 输出相应字符。格式需要与传入的值及其类型相符, 不能仅凭屏幕上想出现什么就随意换转换字母。

下表列出本讲使用的普通字符输出格式。%a、%A 的十六进制浮点形式和宽字符格式不在本讲的使用范围内。

表 3.3: 常用输出转换与相应参数

转换	输出形式	参数说明
%d, %i	有符号十进制整数	int
%u	无符号十进制整数	unsigned int
%o	无符号八进制整数	unsigned int
%x, %X	无符号十六进制整数	unsigned int; 字母分别小写、大写
%f, %F	小数形式浮点数	double; float 值会提升
%e, %E	指数形式浮点数	double; 指数标记分别为 e、E
%g, %G	按数值、精度选择形式	double; 使用有效数字精度
%c	一个字符	int 值, 转换为 unsigned char 后输出
%s	字符串	指向字符序列起点的指针
%p	指针的文本表示	void *, 例如 (void *)&n
%%	百分号	不使用数据参数

void * 是通用对象指针类型; 输出对象地址时, (void *)&n 把变量 n 的地址转换为%p 要求的类型。%p 采用什么具体文本形式由实现规定, 不保证一定像十六进制整数那样显示。

3.5.2 字段宽度规定最少占多少位置

把几本书的页数排成一列, 需要为数字预留位置。这段程序用星号标记输出字段的左右边界, 星号本身不计入宽度:

```
#include <stdio.h>
#define PAGES 931

int main(void) {
    printf("%d*\n", PAGES);
    printf("%2d*\n", PAGES);
    printf("%10d*\n", PAGES);
    printf("%-10d*\n", PAGES);
    return 0;
}
```

```
*931*
*931*
*      931*
*931      *
```

`%2d` 要求字段至少宽 2，931 已经需要 3 个位置，所以三个数字全部输出。`%10d` 还缺 7 个位置，默认在左侧补 7 个空格，使数字右对齐。`%-10d` 中的减号指定左对齐，因此空格补在右侧。

输出字段宽度是最小宽度，不足时补齐，超过时扩展。它不会截去整数的高位，也不会为了塞进指定宽度而改变数值。

3.5.3 同一个精度数字有不同含义

精度写在小数点后，其含义取决于转换字母：

表 3.4: 输出精度的含义

转换	精度控制什么	例子
<code>d, i, u, o, x, X</code>	至少输出多少个数字；不含符号与进制前缀	<code>%.3d</code> 把 6 输出成 006
<code>f, F</code>	小数点后的位数	<code>%.2f</code> 把 12.5 输出成 12.50
<code>e, E</code>	尾数小数点后的位数	<code>%.2e</code> 把 12.5 输出成 1.25e+01
<code>g, G</code>	用于转换的有效数字位数	<code>%.3g</code> 把 12.5 输出成 12.5
<code>s</code>	最多输出多少个内容字节	<code>%.3s</code> 把 Morgan 输出成 Mor

整数精度不足时在数字前补 0；指定精度为 0 且数值为 0 时，可以不输出任何数字。浮点数的 `f`、`e` 默认精度为 6。`g` 默认按 6 位有效数字转换，通常去掉尾部无意义的 0，因此最终打印的数字可以少于 6 位。`%c` 没有精度这一用法。

若 `%g` 的精度为 P ，按该精度舍入后的指数形式的十进制指数记为 X ，当 $X < -4$ 或 $X \geq P$ 时采用指数形式，其他情况采用小数形式。精度写成 0 时按 1 处理。选择形式依据数值与精度，不能简单理解成永远选择字符数最少的一种。

以租金 3852.99 为例，分别改变宽度和精度：

```
#include <stdio.h>

int main(void) {
    const double RENT = 3852.99;

    printf("%f*\n", RENT);
    printf("%e*\n", RENT);
    printf("%.2f*\n", RENT);
    printf("%.1f*\n", RENT);
    printf("%.3f*\n", RENT);
    printf("%.3e*\n", RENT);
    printf("%.2f*\n", RENT);
    printf("%010.2f*\n", RENT);
    return 0;
}
```

采用通常的浮点舍入方式，输出为：

```
*3852.990000*
*3.852990e+03*
*3852.99*
*3853.0*
* 3852.990*
* 3.853e+03*
*+3852.99*
*0003852.99*
```

`%4.2f` 虽然指定宽度 4，但 3852.99 连同小数点需要 7 个位置，仍完整输出。`%3.1f` 先按一位小数舍入为 3853.0，再按实际所需的 6 个位置输出。`%10.3e` 把尾数舍入为 3.853，3.853e+03 占 9 个位置，所以左侧补一个空格。宽度不会减少既定的小数位数。

最后两个调用新增 `+` 和 `0` 标志：正数也显示加号；填充宽度时改用 0。因此，`%010.2f` 在七个位置的 3852.99 之前补三个 0，整个字段宽 10。

3.5.4 标志怎样组合

输出负数时自带负号，正数则可以用加号或一个空格占据符号位置。进制前缀可以用 `#` 补出；对齐和填充由 `-`、`0` 控制。这五种标志各有适用范围：

表 3.5: 输出标志及组合规则

标志	作用
-	在字段内左对齐；省略时通常右对齐。
+	对有符号数值显示正负号，正数和正零也显示加号。
空格	有符号转换的结果没有负号时，前置一个空格；与 + 同时出现时，采用 +。
#	八进制保证首位为 0；非零十六进制整数加 0x 或 0X；浮点输出保留小数点，g/G 还保留尾部的 0。
0	数值输出用 0 填充宽度；与-同时出现时被忽略；整数转换指定精度时也被忽略。

```
#include <stdio.h>

int main(void) {
    printf("%x %X %#x\n", 31u, 31u, 31u);
    printf("**%d**% d**% d**\n", 42, 42, -42);
    printf("**%5d**%5.3d**%05d**\n", 6, 6, 6);
    printf("|%+06d| |%#.0f|\n", 42, 12.0);
    return 0;
}
```

输出为：

```
1f 1F 0x1f
**42** 42**-42**
**      6**  006**00006**
|+00042| |12.|
```

第一行的 31u 是 unsigned int 常量，符合 %x 的类型要求。十进制 31 等于十六进制 1f，%X 只把字母改为大写，%#x 再补上前缀。

第三行比较三个不同要求。%5d 只补空格；%5.3d 先把 6 补成三位数字 006，再在左侧补两个空格；%05d 没有整数精度，五个位置直接用 0 填满。若改成 %05.3d，整数精度使 0 标志失效，结果仍为两个空格加 006。编译器可能对这个多余标志给出警告，实际书写时用 %5.3d 即可。

%+06d 规定总宽度 6，包含一个正号和两个数字，因此补三个 0；零放在符号之后，得到 +00042。%#.0f 没有小数位，但 # 保留小数点，得到 12.。

3.5.5 字符串的宽度与输出上限

字符串输出也可以对齐。以 `Authentic imitation!` 为例，其中包含一个空格，长度为 20：

```
#include <stdio.h>
#define BLURB "Authentic imitation!"

int main(void) {
    printf("%2s/\n", BLURB);
    printf("%24s/\n", BLURB);
    printf("%24.5s/\n", BLURB);
    printf("%-24.5s/\n", BLURB);
    return 0;
}
```

```
/Authentic imitation!/
/   Authentic imitation!/
/                               Authe/
/Authe                          /
```

`%2s` 的宽度小于文字长度，20 个字节仍全部输出。`%24s` 在左边补 4 个空格。`%24.5s` 先把内容限制为前 5 个字节 `Authe`，再补 19 个空格；加上减号后，把 19 个空格放到右侧。

`s` 的精度规定的是输出上限，整数精度规定的是数字个数下限。二者都写在小数点后，却不能交换含义。截短显示也不会修改原数组或原字符串的内容。

对普通`%s`而言，宽度和精度按输出字节处理，不能据此保证中文在终端中整齐对齐。UTF-8 中的一个汉字可能占三个字节，而终端显示宽度又有自己的规则；`%.2s` 甚至可能只输出一个汉字编码的前两个字节。

3.5.6 一条长语句怎样分行书写

源代码的一行太长时，可以在参数之间换行，也可以把长字符串分成相邻的字符串字面量。下面两个字面量在编译时连接，得到一个完整的格式字符串：

```
printf("Name length: %zu bytes; "
       "capacity: %zu bytes.\n",
       strlen(name), sizeof name);
```

第一个字面量末尾保留了空格，所以两段连起来是 `bytes; capacity`。源代码中的换行和缩进不进入字符串，只有写在引号内的`\n`才要求输出换行。

也可以用两次调用输出同一行：

```
printf("Here's one way to print a ");  
printf("long string.\n");
```

第一次调用没有输出换行，第二次便接在后面。较早的 C 程序中还可以见到行末反斜杠续行：

```
printf("Here's another way to print a \  
long string.\n");
```

反斜杠必须紧接源文件的换行，预处理时二者一起被删除；下一行开头若有缩进空格，那些空格会留在字符串里。相邻字面量的写法更便于缩进，也更容易看清最终文字中的空格。

3.6 格式、类型与返回值

3.6.1 长度修饰符并不规定固定字节数

%ld 中的 l 说明参数按 long 转换，%Lf 中的大写 L 说明参数为 long double。这些**长度修饰符**选择类型，实际字节数仍由该类型在当前实现中的大小决定。

把前面出现的各部分放在一起，输出转换说明的书写顺序是：

```
%[flags][width][.precision][length]specifier
```

这是格式结构的示意，方括号表示可省略的部分，不写进实际格式。flags 是对齐、符号、前缀或填充标志，width 是最小宽度，precision 是精度，length 是长度修饰符，末尾的 specifier 是转换字母。例如，%-10.3f 按顺序写左对齐标志、宽度 10、精度 3 和浮点转换字母 f，没有长度修饰符。

表 3.6: 常用整数与浮点类型的输出格式

原对象类型	常用格式	传参与转换
signed char	%hhd	经整数提升传入，再按有符号字符类型转换
unsigned char	%hhu	经整数提升传入，再按无符号字符类型转换
short	%hd	经整数提升传入，再按 short 转换
unsigned short	%hu	经整数提升传入，再按 unsigned short 转换
int	%d	传入 int 值
unsigned int	%u	传入 unsigned int 值
long	%ld	传入 long 值
unsigned long	%lu	传入 unsigned long 值
long long	%lld	传入 long long 值
unsigned long long	%llu	传入 unsigned long long 值
size_t	%zu	传入大小类型的值
float, double	%f	float 先提升为 double
long double	%Lf	传入 long double 值

hh、h 与 l、ll 各自选择相应整数类型，z 选择大小类型。在输出浮点数时，%lf 与%f 含义相同；这一点不能移用到 scanf，输入 float 和 double 需要区分%f 与%lf。

3.6.2 同一个整数为什么能显示成字符

%c 要求一个 int 值，再把它转换为 unsigned char 用于输出。在 8 位字节、ASCII 环境中，336 转换为 unsigned char 后等于 $336 \bmod 256 = 80$ ，编码 80 对应 P。整个 8 位字节都参与转换，不能把它说成“只看低 7 位”；低 7 位恰好足以表示 ASCII 基本字符，并不改变%c 的转换规则。

下面对同一组数值作几种明确的类型转换。给出输出时，另假定 short 为 16 位，int 为 32 位，且当前实现把超出 short 范围的整数转换为保留低 16 位的相应有符号值。

```
#include <stdio.h>

int main(void) {
    short num = 336;
    short mnum = -336;
    long words = 65618L;
    unsigned short wrapped = (unsigned short)mnum;

    printf("short: %hd; unsigned short: %hu\n",
           mnum, wrapped);
    printf("number: %d; character: %c\n", (int)num, num);
    printf("words: %ld; short: %hd; character: %c\n",
           words, (short)words, (int)(unsigned char)words);
    return 0;
}
```

```
short: -336; unsigned short: 65200
number: 336; character: P
words: 65618; short: 82; character: R
```

转成 16 位 `unsigned short` 时按模 2^{16} 计算： $-336 + 65536 = 65200$ 。这个无符号值由 `wrapped` 保存，`%hu` 把它输出成 65200。原变量 `mnum` 仍为 -336 。

$65618 = 65536 + 82$ ，所以在已指定的 `short` 转换规则下，`(short)words` 得到 82。转成 8 位 `unsigned char` 则按模 256 得到 82，ASCII 编码 82 对应 R。前者的超范围有符号转换在 C11 中由实现规定，后者的无符号转换遵守模运算规则；这两种依据不同。

若希望查看负整数转换后的十六进制形式，可以明确写出 `(unsigned int)mnum`，再用 `%x` 输出。这个转换按 `unsigned int` 的范围得到一个无符号值，输出的是该值的十六进制形式。`%x` 不负责为负数打印一个负号，也不应被当作任意类型的内存查看器。

3.6.3 缺参数与错类型没有可靠的预测输出

以下是用于诊断的错误片段，不应靠运行结果判断哪个写法“似乎也能用”：

```
int i = 7, j = 8;
double x = 3.5;

printf("%d %d\n", i);
printf("%d\n", x);
printf("%d\n", i, j);
```

第一个调用要求两个 `int` 参数，只提供了一个，导致未定义行为。第二个调用让 `%d` 对应 `double`，同样导致未定义行为；不能解释成把 3.5 自动变成 3。若确实需要整数 3，应先显式转换，再输出：`printf("%d\n", (int)x);`。

第三个调用多传了 `j`，它仍会被求值，但没有对应的转换说明，输出函数会忽略这个多余参数，结果为 7 后跟换行。它与前两个调用的性质不同，仍应删除多余参数，使对应关系清楚。

当格式字符串直接写在源代码中时，编译器通常能发现这类问题。用 GCC 编译可开启 `-Wall -Wextra -Wformat=2` 检查；诊断能力有范围限制，程序员仍需核对转换说明、参数顺序和类型。没有警告不能证明所有动态构造的格式都正确。

3.6.4 printf 返回多少

`printf` 除了输出文字，还返回一个 `int`。输出成功时，返回实际输出的字符数，包括空格和换行；输出发生错误时返回负值。下面按通常以华氏 212 度表示水沸点的例子，统计一句英文的输出长度：

```
#include <stdio.h>

int main(void) {
    int boiling = 212;
    int rv;

    rv = printf("%d F is water's boiling point.\n", boiling);
    printf("printf returned %d.\n", rv);
    return 0;
}
```

```
212 F is water's boiling point.
printf returned 32.
```

第一行在换行前有 31 个字符，换行再计 1，因此返回 32。格式串里的 `%d` 本身占两

个源代码字符，输出时却变成三个字符 212，计数依据实际输出。字符串末尾的空字符不被输出，也不计数。

在 Windows 文本输出中，一个 `\n` 可能被底层转成回车与换行两个文件字节；`printf` 仍把此次换行算作一个输出字符。函数的返回值与操作系统最终文件大小，不能在所有环境中直接等同。输出 UTF-8 文字时，普通窄字符输出的计数也不等于可见汉字个数。

3.7 格式化输入的读取过程

3.7.1 格式说明、地址与赋值

键盘输入 18 20000 llama，程序需要把 18 保存为年龄，把 20000 保存为浮点金额，把 llama 保存为宠物名称。一次读取可以包含多种转换：

```
#include <stdio.h>

int main(void) {
    int age;
    float assets;
    char pet[30];

    printf("Enter your age, assets, and favorite pet.\n");
    if (scanf("%d%f%29s", &age, &assets, pet) != 3) {
        printf("Invalid input.\n");
        return 1;
    }
    printf("%d $%.2f %s\n", age, assets, pet);
    return 0;
}
```

输入上述内容后，输出 18 \$20000.00 llama。输入也可以分成三行，因为这三种转换都会先跳过数据前的空白。格式字符串中没有写空格，并不意味着使用者必须把年龄、金额与名字连在一起。

表 3.7: 输入目标与相应的转换、参数

定义	转换	对应参数
int n	%d	&n
unsigned int n	%u, %o, %x	&n
short n	%hd	&n
unsigned short n	%hu	&n
long n	%ld	&n
long long n	%lld	&n
size_t n	%zu	&n
float x	%f, %e, %g	&x
double x	%lf, %le, %lg	&x
long double x	%Lf, %Le, %Lg	&x
char ch	%c	&ch
char word[30]	%29s	word

输入 float 时, %f 和 %e 都能接受小数或指数形式; 它们不会像输出那样要求分别使用某一种外观。double 则必须加 l, 因为地址指向的对象已经有确定类型, 地址参数不会发生 float 到 double 的值提升。

%d 接受可带正负号的十进制整数。%i 在输入中依据前缀判断进制, 例如 010 按八进制得到 8, 0x10 按十六进制得到 16; %d 读取 010 则得到十进制 10。输出时 %d 和 %i 等价, 输入时二者有这一区别。

3.7.2 一次成功赋值后, 后面还有什么

假设每次都从同样的输入开头重新读取:

```
-13.45e12# 0
```

表 3.8: 不同转换在同一输入中的停止位置

转换	赋给目标的内容	随后尚未读取的开头
%d	整数 -13	.45e12# 0
%lf	浮点数 -13.45×10^{12} 的表示	# 0
%19s	字符串 -13.45e12#	空格, 随后是 0

表中的三种转换是三次独立实验, 并未依次作用于同一条输入。若在每次转换后紧

接一次%c，读到的字符依次为小数点、# 和空格。这些字符都因不属于前一种转换要接收的内容而被留下。

scanf 的返回值统计**成功转换并赋值的项目数**。普通格式字符的匹配、跳过空白和稍后介绍的抑制赋值，都不增加这个计数。

```
#include <stdio.h>

int main(void) {
    int n = -1;
    double x = -1.0;
    int count = scanf("%d %lf", &n, &x);

    printf("count=%d, n=%d, x=%.1f\n", count, n, x);
    return 0;
}
```

为了观察失败后的状态，程序先把两个变量都初始化为 -1，再尝试输入。每次重新运行，分别提供以下内容：

表 3.9: 输入返回值与变量状态

输入	返回值	n	x
12 3.3	2	12	3.3
12 abc	1	12	-1.0（原值）
abc	0	-1（原值）	-1.0（原值）
开始即输入结束	EOF	-1（原值）	-1.0（原值）
12 3.3abc	2	12	3.3

12 abc 在第二次转换处失败，第一个整数转换已经完成，不会因为后面失败而撤销。abc 在第一次数值转换处就不匹配，因此返回 0。若第一次转换完成前发生输入结束或读取错误，返回 EOF；EOF 是 end of file 的缩写，在 <stdio.h> 中定义为一个负整数，不应把必须等于 -1 写进程序逻辑。

12 3.3abc 的两个数值转换都成功，返回 2，尾部 abc 仍未读取。成功项目数只保证相应变量获得值，不能证明整行都符合输入规则。再次用数值转换读取同一个未读字母时，还会失败；重复调用本身不会使错误字符消失。

本讲数值示例假定数值在目标类型可表示范围内。超范围的数值转换不能仅靠返回值获得可靠的范围错误报告。需要接受任意文本并严格检查整行的程序，可以先读取整行，再使用专门的数值转换函数逐项检查。

3.7.3 普通字符必须在相应位置匹配

格式字符串中的逗号是一个真实的匹配要求：

```
int n, m;  
int count = scanf("%d, %d", &n, &m);
```

输入 88, 121, 两个转换成功, 返回 2。输入 88 , 121, 第一个%d 在空格前停止, 接下来格式要求逗号, 却在输入中看见空格, 匹配失败, 返回 1。格式中逗号后面的空格没有机会替逗号前面跳过空白。

如果希望允许逗号前后有空白, 应写:

```
int count = scanf("%d , %d", &n, &m);
```

这条语句是前一写法的替代版本。格式中任意一个空白字符都要求跳过输入中连续的空白, 数量可以是零。因此它可以接收 88,121、88 , 121, 也可以接收逗号前后换行的输入。

常见输入空白包括空格、水平制表和换行; 标准 C 还把回车、换页和垂直制表列为空白。在本讲使用的数值转换与%s 之前, 前导空白会自动被跳过。%c 不跳过; 扫描字符集合的%[同样不跳过, 记录已读取数量的%n 也不执行通常的数据读取。这两个扩展格式不用于本讲示例, 因而不能把规则概括为“除%c 外全部跳过”。

3.7.4 字符输入为什么会读到换行

输入整数后按 Enter, 换行也是输入内容的一部分。%d 只读取整数所需的字符, 留下换行; 紧接着的%c 便可以立即读到它。

```
#include <stdio.h>

int main(void) {
    int n;
    char ch;

    if (scanf("%d", &n) != 1) {
        return 1;
    }
    if (scanf("%c", &ch) != 1) {
        return 1;
    }
    printf("n=%d, next code=%d\n", n, (unsigned char)ch);
    return 0;
}
```

在 ASCII 环境下，输入 12 后按 Enter，输出为 `n=12, next code=10`；10 是换行的编码。若输入 12 A 后按 Enter，`%c` 先读到空格，编码变为 32，字母 A 仍未读取。

想读取下一个非空白字符，可以把字符输入语句改为 `scanf(" %c", &ch)`。前导空格先跳过输入中的连续空白，`%c` 再读一个字符。它会跳过多少空白取决于输入，不能把这个空格理解为“只吃掉一个换行”。若本来就想检查空格或换行，则保留不带前导空格的`%c`。

同样的规则说明了为什么不应为普通整数输入随手加上结尾换行：`scanf("%d\n", &n)` 读完整数后还要继续跳过空白，可能等到下一个非空白字符到达才返回。格式字符串的`\n`没有“本次输入到 Enter 为止”的含义。

3.7.5 输入宽度规定最多接收多少

`printf` 的宽度是最小输出宽度，`scanf` 的宽度则限制本次转换最多从输入项读取多少个字符。转换开始前跳过的空白不计入这个上限，数字的符号则计入数值输入项的宽度。

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char word[5];
    char next;

    if (scanf("%4s", word) != 1) {
        return 1;
    }
    if (scanf("%c", &next) != 1) {
        return 1;
    }
    printf("word=[%s], length=%zu, next=[%c]\n",
        word, strlen(word), next);
    return 0;
}
```

输入 ABCDE 后按 Enter，输出：

```
word=[ABCD], length=4, next=[E]
```

%4s 写入 A、B、C、D，再额外写一个空字符，正好占满 5 字节。E 没有被吞掉，由随后的%c 读取，换行仍留在输入中。%4s 的 4 不包括自动写入的结束字符。

%c 也可以带宽度，但不会自动写空字符。例如，char pair[2]; scanf("%2c", pair); 用于读取两个原始字符，不能随后把 pair 当作以空字符结尾的字符串输出。输入宽度只限制本次转换，不会替程序检查后续输入是否合理。

需要读含空格的一行时（选读）

fgets 用于读取一段行文本。例如，对 char line[40] 调用 fgets(line, sizeof line, stdin)，成功时最多读入 39 个字符，再补空字符；遇到换行会把换行一起保存后停止。stdin 是标准输入流，通常连接键盘。函数返回数组首地址表示成功；失败或尚未读入字符就遇到输入结束时返回空指针，可与 NULL 比较。空指针表示没有有效对象地址，和字符串内容中的空字符是不同概念。

若一行超过容量，一次调用只能取得前一部分，不能据成功返回就认定整行已经读完。与 scanf 混用时，还需检查前一次转换留下的换行是否会被 fgets 立即读到。标准 C 没有用 fflush(stdin) 清空这些字符的通用规则。

3.8 星号在输入与输出中各做什么

3.8.1 输出时由参数提供宽度和精度

已经确定的列宽可以直接写成`%8d`。如果列宽由使用者选择，`printf` 允许在宽度位置写星号，从后面的参数取得宽度。精度也可以如此指定：

```
printf("|%*. *f|\n", width, precision, value);
```

第一个星号使用 `width`，第二个使用 `precision`，`f` 使用 `value`。两个星号各要求一个 `int` 参数，最后的浮点转换要求 `double`。宽度与精度是整数参数，不能为了表示“通常非负”就把它们随意改成 `unsigned int`。

```
#include <stdio.h>

int main(void) {
    int width, precision;
    int number = 256;
    double weight = 242.5;

    printf("What field width?\n");
    if (scanf("%d", &width) != 1) {
        return 1;
    }
    printf("The number is: |%*d|\n", width, number);
    printf("Now enter a width and a precision:\n");
    if (scanf("%d%d", &width, &precision) != 2) {
        return 1;
    }
    printf("Weight = |%*. *f|\n", width, precision, weight);
    return 0;
}
```

第一次输入 8，第二次输入 8 2，两个结果字段为：

```
The number is: |      256|
Weight = |  242.50|
```

宽度给成负值时，按其绝对值确定宽度并左对齐。例如宽度 `-8` 使 256 右侧补 5 个空格。用星号提供的精度若为负，按省略精度处理，`f` 恢复默认的六位小数。本例练习使用幅度较小的宽度和精度，以便观察空格；过大的参数会产生大量输出。

3.8.2 输入时跳过项目而不赋值

有三项输入，只需要保存最后一项时，可以用 `scanf` 的 `*` 抑制赋值。抑制赋值表示仍按对应规则读取和转换输入项，但不把结果保存到变量中：

```
#include <stdio.h>

int main(void) {
    int n;

    printf("Please enter three integers:\n");
    if (scanf("%*d%*d%d", &n) != 1) {
        return 1;
    }
    printf("The last integer was %d.\n", n);
    return 0;
}
```

输入 2013 2014 2015，最后输出 The last integer was 2015.。前两项不对应地址参数，第三项才对应 `&n`；真正赋值只有一次，所以应检查返回值为 1，而不是 3。

如果输入 2013 abc 2015，第二个 `%*d` 仍然要求一个整数，在字母 a 处失败，后面的 2015 不会被读取。抑制赋值不表示“忽略任意一段文字”。这次返回 0，因为前一项虽然已成功转换，但被抑制，没有产生赋值；程序结束而不读取未初始化的 `n`。

表 3.10: 两个函数中星号的不同含义

写法	星号的作用	相应参数
<code>printf("%*d", w, n)</code>	宽度由参数取得	<code>w</code> 与 <code>n</code> 均为 <code>int</code>
<code>printf("%.*f", p, x)</code>	精度由参数取得	<code>int</code> 精度、 <code>double</code> 值
<code>scanf("%*d%d", &n)</code>	第一项读取后不赋值	仅传第二项的地址

`scanf` 没有把 `%*s` 解释成动态字符串宽度的规则；这里的星号仍表示抑制赋值。输入字符串要写 `%39s` 这样的明确上限，或使用能够接收缓冲区大小的读行函数。

3.9 一次只读写一个字符

3.9.1 getchar 的返回值为什么用 int 接收

`getchar()` 从标准输入读取下一个字符，不需要参数；成功时返回所读字节按 `unsigned char` 解释后转换成的 `int` 值，输入结束或读取错误时返回 `EOF`。`putchar(ch)` 把一个字符写到标准输出，成功时返回所写字符的值，输出失败时返回 `EOF`。

```
#include <stdio.h>

int main(void) {
    int ch;

    printf("Enter one character:\n");
    ch = getchar();
    if (ch == EOF) {
        return 1;
    }
    printf("You entered: ");
    if (putchar(ch) == EOF) {
        return 1;
    }
    putchar('\n');
    return 0;
}
```

输入 A 后按 Enter，程序输出 You entered: A。输入中的换行还没有被 `getchar` 读取。通常终端会先收集一行键盘输入，所以 `getchar` 不一定在按下 A 的瞬间就返回；“读取一个字符”描述的是这次调用取得的内容，不规定终端的按键交付方式。

在常见的 8 位字节环境中，一个输入字节的值可以是 0 至 255。接收变量还必须保留额外的失败标记 `EOF`，因此应使用 `int`。若先存入 `char`，再与 `EOF` 比较，可能在转换时丢失二者的区别。`EOF` 是函数报告的状态值，不是文件末尾存放的一个特殊字节；空字符 `'\0'` 则是可以真实读到的数据值 0。

3.9.2 与格式化字符读写的对应

`getchar()` 和 `scanf("%c", &ch)` 都可以读取一个普通字符，也都不会自动跳过空格或换行。但接口不同：前者直接返回字符或 `EOF`，后者把字符写入 `char` 对象，并返回成功赋值项数或 `EOF`。不能把两者的返回值当成同一种数据使用。

`putchar(ch)` 与 `printf("%c", ch)` 都可以输出一个字符。前者成功时返回字符值，后者成功时返回输出数量 1。只处理一个字符时，专门的字符函数调用更直接；性能取决于具体实现和缓冲方式，不能仅凭函数名称保证一定更快。

历史旁注

汤普森的 B 语言手册已经列出 `getchar`、`putchar` 和 `printf`。字符函数从标准输入取得字符，或向标准输出写字符；格式输出则在文字中解释转换指令。今天的 C 库保留了这些熟悉的名字，但具体接口和错误报告应按 C 的规则使用。程序既可能连接终端，也可能读文件、写文件；把数据来源交给标准输入输出接口处理，程序便不必把每个读写动作都写成键盘和屏幕的专用操作。

`getchar` 与 `putchar` 都是标准库函数，由 `<stdio.h>` 提供声明。实现还可以在头文件中提供同名的函数式宏，即写成函数调用形式的宏，以方便优化。这是实现办法，不表示它们“并非真正的函数”。本讲直接按规定的函数接口调用即可。

3.10 习题与解析

判断输出时，先确定实参类型，再按精度生成要显示的文字，最后按宽度补齐空格或 0。判断输入时，从尚未读取的第一个字符开始，逐项记录是否跳过空白、是否匹配、是否赋值以及在哪里停止。

3.10.1 习题一：预测文字、字符和数字的输出

在 ASCII 环境中，写出以下程序的输出，并说明字符串长度和指数形式的小数位数怎样得到。

```
#include <stdio.h>
#include <string.h>
#define Q "His Hamlet was funny without being vulgar."

int main(void) {
    printf("He sold the painting for $%2.2f.\n", 2.345e2);
    printf("%c%c%c\n", 'H', 105, '\41');
    printf("\'%s'\nhas %zu characters.\n", Q, strlen(Q));
    printf("Is %2.2e the same as %2.2f?\n", 1201.0, 1201.0);
    return 0;
}
```

解析。在通常的浮点舍入环境中，程序输出：

```

He sold the painting for $234.50.
Hi!
"His Hamlet was funny without being vulgar."
has 42 characters.
Is 1.20e+03 the same as 1201.00?

```

2.345e2 表示 234.5, %2.2f 要求小数点后两位, 得到 234.50。宽度 2 容纳不下整个结果, 字段自动扩展; 句末的句点是格式字符串中的普通字符。

'H' 输出字母 H, ASCII 编码 105 输出 i。'\41' 中的 41 按八进制解释, $4 \times 8 + 1 = 33$, ASCII 编码 33 对应感叹号。

Q 中的文字按字节计数: His 为 3, Hamlet 为 6, was 为 3, funny 为 5, without 为 7, being 为 5, vulgar. 为 7, 加上六个空格, 共 $3 + 6 + 3 + 5 + 7 + 5 + 7 + 6 = 42$ 。外侧双引号由格式串中的\" 另外输出, 不属于 Q, 也不计入 strlen(Q)。

%2.2e 把 1201.0 显示成尾数保留两位小数的 1.20e+03; %2.2f 则显示成 1201.00。指数形式在此只保留了三位有效数字, 所以从显示出的文字还原数值时, 前者为 1200, 后者为 1201。传入的原值相同, 显示舍入后表达的精度不同。某些运行库的指数可能显示为 e+003, 不能把指数前导零的数量当作数值差别。

3.10.2 习题二：修正姓名和年龄程序

下面是故意写错的程序。它原本想读入名字和年龄, 把年龄加 10 后输出一句玩笑话。找出语法、存储类型、格式和参数的问题。

```

define B booboo
define X 10
main(int) {
    int age;
    char name;

    printf("Please enter your first name. ");
    scanf("%s", &name);
    printf("All right, %c, what's your age?\n", name);
    scanf("%f", age);
    xp = age + X;
    printf("That's a %s! You must be at least %d.\n", B, xp);
    rerun 0;
}

```

解析。两个 `define` 前缺少 `#`，`booboo` 需要双引号才能成为文字。还须补上 `<stdio.h>`，把函数头改成 `int main(void)`，声明 `xp`，把 `rerun` 改成 `return`。

`char name` 只有一个字符的空间，连单字母名字的内容与结束字符都装不下，应改成字符数组。读取数组使用有限宽度的 `%s`，输出名字也用 `%s`。年龄是 `int`，读取时用 `%d` 并传入 `&age`。在年龄为 0 至 150 的练习输入范围内，加 10 能够安全地保存在 `int` 中：

```
#include <stdio.h>
#define B "booboo"
#define X 10

int main(void) {
    int age, xp;
    char name[40];

    printf("Please enter your first name.\n");
    if (scanf("%39s", name) != 1) {
        return 1;
    }
    printf("All right, %s, what's your age?\n", name);
    if (scanf("%d", &age) != 1) {
        return 1;
    }
    xp = age + X;
    printf("That's a %s! You must be at least %d.\n", B, xp);
    return 0;
}
```

输入 `Alice` 和 `18` 后，最后输出 `That's a booboo! You must be at least 28.`。数组容量、输入上限、字符串格式与年龄地址必须同时正确，仅把程序改到能够编译还不够。

3.10.3 习题三：补出带引号和百分号的输出

给定书名 `War and Peace`、价格 `12.99` 和百分数 `80.0`，补出两条 `printf` 语句，使程序输出：

```
This copy of "War and Peace" sells for $12.99.
That is 80% of list.
```

解析。完整程序为：

```
#include <stdio.h>
#define BOOK "War and Peace"

int main(void) {
    float cost = 12.99f;
    float percent = 80.0f;

    printf("This copy of \"%s\" sells for $%.2f.\n",
           BOOK, cost);
    printf("That is %.0f%% of list.\n", percent);
    return 0;
}
```

\ " 在字符串中表示一个要输出的双引号，%% 在格式字符串中表示一个要输出的百分号。两者分别作用于字符串的书写规则和格式转换规则，不能把百分号写成 \% 来代替%%。

%.2f 给价格保留两位小数，%.0f 不给百分数输出小数位。美元符号 \$ 是 C 字符串中的普通字符，不需要特殊转换说明。

3.10.4 习题四：由显示要求写出转换说明

为下列十五项要求选择转换说明。题中的宽度均指最小字段宽度；十六进制、八进制实参采用相应的无符号整数类型，普通小数实参为 double，字符串只含基本英文字符。

1. 十进制整数，按实际需要的宽度输出。
2. 十六进制整数，用大写字母，例如 6A，字段宽度 4。
3. 浮点数，显示为 232.445 这样的形式，字段宽度 10。
4. 浮点数，显示为 2.45e+02 这样的形式，字段宽度 12。
5. 字符串，字段宽度 30，左对齐。
6. unsigned long 整数，字段宽度 15。
7. 十六进制整数，显示为 0x6a 这样的形式，字段宽度 4。
8. 正数显示加号，如 +232.445，字段宽度 10。
9. 浮点数，显示为 2.45E+02 这样的形式，字段宽度 12，左对齐。

10. 最多输出字符串前 8 个字节，字段宽度 8。
11. 十进制整数，至少 4 个数字，字段宽度 6。
12. 八进制整数，字段宽度由前一个参数提供。
13. 一个字符，字段宽度 2。
14. 浮点数，显示为 +3.13 这样的形式，不额外指定宽度。
15. 最多输出字符串前 5 个字节，字段宽度 7，左对齐。

解析。各项答案及检验例子如下。例子在字段外加竖线，空格位于竖线之内。

表 3.11: 十五项输出要求的转换说明与结果

序号	转换说明	给定参数时的字段
1	%d	931: 931
2	%4X	106u: 6A
3	%10.3f	232.445: 232.445
4	%12.2e	245.0: 2.45e+02
5	%-30s	C, 后接 29 个空格
6	%15lu	123UL: 12 个空格后接 123
7	%#4x	106u: 0x6a
8	%+10.3f	232.445: +232.445
9	%-12.2E	245.0: 2.45E+02
10	%8.8s	ABCDEFGH IJ: ABCDEFGH
11	%6.4d	23: 0023
12	%*o	宽度 5、值 10u: 12
13	%2c	'A': A
14	%+.2f	3.13: +3.13
15	%-7.5s	Morgan: Morga

第 14 项要求显示两位小数并带符号，写出 + 和 .2 即可，无须增加最小字段宽度。数字本身需要的所有位置都会保留。

第 4、9 项的指数至少占两位，若某实现多输出一个指数前导零，字段中的补齐空格相应减少。第 10、15 项只限制最多输出的字节数；字符串比这个上限短时，遇到空字符就停止，再按字段宽度补空格。

3.10.5 习题五：为输入选择对象和地址

分别为以下四组输入写出变量定义与 `scanf` 调用：整数 101；两个浮点数 22.32 8.34E-09；单词 `linguini`；单词与整数 `catch 22`。

解析。四段代码分别使用，各次输入前变量按对应片段定义：

```
int n;
if (scanf("%d", &n) != 1) {
    return 1;
}
```

```
float x, y;
if (scanf("%f%f", &x, &y) != 2) {
    return 1;
}
```

```
char word[20];
if (scanf("%19s", word) != 1) {
    return 1;
}
```

```
char word[20];
int value;
if (scanf("%19s%d", word, &value) != 2) {
    return 1;
}
```

两项浮点输入都用 `%f`，因为小数形式和指数形式都可以转换到 `float`。如果变量改成 `double`，两个格式也要对应改成 `%lf`。最后一组输入中，`%19s` 在 `catch` 之后的空格前停止，`%d` 跳过这个空格并读取 22。

3.10.6 习题六：数组容量、内容和终止位置

给定以下独立定义，写出数组大小和能够合法调用的 `strlen` 结果，并指出不能当作完整字符串使用的数组：

```
char a[8] = "Hi";
char b[] = "Hi\n";
char c[] = "AB\0CD";
char d[2] = {'H', 'i'};
char e[] = "";
```

解析。结果分别为：a 大小 8、长度 2；b 大小 4、长度 3；c 大小 6、长度 2；d 大小 2，但没有字符串结束字符，不能对它调用 `strlen`；e 大小 1、长度 0。

若随后执行 `a[1] = '\0'`；a 的长度变为 1，大小仍为 8。若另定义 `char f[2] = "Hi"`；C 允许恰好装入两个字母而省去放不下的结束字符，但 `f` 同样不能当作以空字符结束的字符串。定义能够通过编译与内容满足字符串约定，是两项不同的检查。

3.10.7 习题七：宽度和精度和补零怎样配合

分别写出下列片段的输出，并说明补空格或补 0 的数量：

```
printf("|%6.4d|\n", -23);
printf("|%06d|\n", -23);
printf("|%-7.5s|\n", "Morgan");
printf("|%.0d|\n", 0);
```

解析。输出为：

```
| -0023|
|-00023|
|Morga  |
||
```

`%6.4d` 要求至少 4 个数字，不把负号计入数字个数。23 先补成 0023，加负号后共 5 个位置，所以还需补一个空格。`%06d` 要求总宽度 6，负号和 23 共占 3 个位置，补三个 0；0 写在负号后面。

`%-7.5s` 最多输出 5 个字节 `Morga`，右侧补两个空格。`%.0d` 对值 0 不输出数字，故只剩格式串里的两条竖线。

3.10.8 习题八：在输入中标出停止位置

每次都重新初始化 `int a = -1, b = -1`；分析 `scanf("%d, %d", &a, &b)` 对以下四种输入的返回值、变量结果及未读部分。各输入末尾都有换行。

1. 88, 121
2. 88 , 121

3. 88, abc

4. 88, 121xyz

解析。第一种返回 2, a=88、b=121, 留下换行。第二种返回 1, a=88、b=-1, 从 88 后的空格开始都还没有读取, 因为格式要求立即匹配逗号。

第三种已匹配逗号并跳过空格, 但第二个%d 在 a 处失败, 返回 1, b 保留原值 -1, 留下 abc 及换行。第四种返回 2, 两个变量均成功赋值, 留下 xyz 及换行。

若把格式改成"%d , %d", 第二种输入也能成功, 因为逗号前新增的格式空格会先跳过输入空格。若随后要读一个非空白字符, 可以用" %c"; 这不会使 xyz 凭空消失, 仍会先读到 x。

3.10.9 习题九：两个星号和一个返回值

考虑以下两条独立调用：

```
printf("|%*. *f|\n", 8, 2, 12.5);
int count = scanf("%*d%4s", word);
```

其中 word 定义为 char word[5];。第一条需要几个数据参数, 输出什么? 第二条输入 10 ABCDE 后按 Enter, count 为多少, word 保存什么, 下一次 getchar() 会取得什么?

解析。第一条需要三个数据参数, 分别为 int 宽度 8、int 精度 2 和 double 值 12.5, 输出| 12.50|后换行。

第二条先读取整数 10 而不保存, 再跳过空格, 最多读取四个非空白字符, 写入 ABCD 和空字符。只有字符串赋值被计数, 所以 count=1。字母 E 还未读取, 下一次 getchar() 取得 E。若给 scanf 额外传入一个“宽度变量”, 它不会替代格式中的 4, 因为输入函数的星号不表示动态宽度。

3.10.10 习题十：逐项检查接口是否匹配

下列各项分别说明是正确用法、违反语言约束、未定义行为, 还是需要修正但具有规定行为的多余参数。各行单独考察; s 已由 char s[8] = "Hi"; 定义, x 是 double, n 是 int。

```
printf("%zu\n", strlen(s));
printf("%d\n", 3.5);
printf("%d\n", 3, 4);
scanf("%f", &x);
scanf("%7s", s);
s = "Hello";
printf("%s\n", s);
```

解析。第一项在包含 `<string.h>` 和 `<stdio.h>` 的前提下正确，输出 2。第二项用 `%d` 接收 `double`，导致未定义行为。第三项输出 3 并忽略多余的 4，应删除 4 以保持清楚。

第四项把 `double` 地址交给要求 `float` 地址的 `%f`，若发生相应写入，行为未定义，应改成 `%lf`。第五项的字符数组地址和宽度搭配正确，还应检查返回值是否等于 1。第六项对数组整体赋值违反语言约束，不能把初始化规则用于这条赋值语句。第七项对独立考察时已初始化的 `s` 正确，输出 Hi。

最后比较两种接收字符的写法。`char ch; scanf("%c", &ch);` 使用字符对象接收数据，再检查输入函数的返回值；`int ch = getchar();` 则把字符值或 EOF 直接存入整数对象。按各自接口检查成功后，才使用所读字符。

3.11 本讲总结

40 字节的名字数组保存 Morgan 时，`sizeof` 得到 40，`strlen` 得到 6。六个字母之后还需要一个空字符，输出 `%s` 与长度函数都按这个结束约定处理内容。向数组写入字符串时，容量必须覆盖内容及结束字符；空格、换行和 UTF-8 的多个编码字节也都是真实的数据。

宏让常用的数值或文字在预处理时按名字展开，`const` 则定义有类型的只读对象。使用宏时，可以把展开后的表达式写出来检查等号、分号与运算顺序；使用 `const` 时，仍要区分只读对象与 C 语言规定的常量表达式。

`printf` 先依照转换字母及精度生成文字，再按字段宽度与标志安排显示。输出宽度是下限，字符串精度是内容字节数的上限；整数精度又表示数字个数的下限。格式还规定了参数类型与数量，少参数或错类型不能靠猜测某次输出解释。

`scanf` 沿输入逐项读取，把转换结果写入指定对象。数值前的空白可以被跳过，普通格式字符必须匹配，`%c` 则直接接收下一个字符。每次调用结束后，都可以继续问三个具体问题：哪些变量得到了新值，输入读取到哪里，下一个字符是什么。返回值报告成功赋值的数量，无法代替对整行内容的检查。

`getchar` 用 `int` 返回字符或 EOF，`putchar` 输出一个字符。它们和格式化输入输出共享标准输入、标准输出，也共享尚未读取的字符。明确每次操作的对象、容量、类型与停止位置，才能使姓名输入、数值计算和结果显示按预期衔接。